

Part 01

1-Question: Why is it better to code against an interface rather than a concrete class?

A: Coding against an interface decouples your code from specific implementations, allowing you to swap out classes easily (loose coupling), making the system much more flexible, testable, and maintainable.

2-Question: When should you prefer an abstract class over an interface?

A: You prefer an abstract class when you want to share common code, fields, and default behavior among closely related classes, or when you need to use non-public access modifiers for shared members.

3-Question: How does implementing `Comparable` improve flexibility in sorting?

A: It allows custom objects to define their own natural ordering logic. This lets built-in sorting functions like `Array.Sort()` or `List<T>.Sort()` sort complex custom objects automatically without needing manual comparison logic.

4-Question: What is the primary purpose of a copy constructor in C#?

A: Its primary purpose is to create a brand-new object by copying the exact data values from an existing object of the same type, ensuring a reliable deep copy when needed.

5-Question: How does explicit interface implementation help in resolving naming conflicts?

A: If a class implements two different interfaces that contain a method with the exact same name, explicit interface implementation allows you to separate them by prefixing the method name with the interface name, preventing naming collisions.

6-Question: What is the key difference between encapsulation in structs and classes?

A: Structs are value types allocated on the stack, meaning copying a struct copies all its encapsulated data directly. Classes are reference types on the heap, where encapsulation protects fields behind references without duplicating the underlying heap data during simple assignments.

7-Question: What is abstraction as a guideline, what's its relation with encapsulation?

A:

Abstraction focuses on *what* an object does by hiding complex implementation details behind a clean interface or abstract class.

Encapsulation focuses on *how* it is implemented by bundling data and restricting direct access to it. Abstraction defines the outer behavior model, while encapsulation protects the inner data supporting it.

8-Question: How do default interface implementations affect backward compatibility in C#?

A: They improve backward compatibility by allowing developers to add new methods with default bodies to an existing interface without breaking all the older classes that already implement that interface.

9-Question: How does constructor overloading improve class usability?

A: It gives users of the class multiple flexible ways to initialize an object depending on what data is available at creation time (e.g., creating a book with just a title, or with both title and author).

Part 02 Reports & Explanations

1- LinkedIn article about abstract class?

A: *Ever wondered how large software systems enforce a strict architectural structure across different teams? The answer is Abstract Classes!*  An abstract class serves as a master blueprint. You cannot instantiate it directly using `new`—instead, it is designed to be inherited. It can contain both fully implemented methods (shared code) and abstract methods (missing bodies that derived classes *must* implement). Using abstract classes prevents code duplication and enforces a consistent contract across related object hierarchies. #CSharp #DotNet #SoftwareArchitecture #OOP

2- What we mean by coding against interface rather than class? and if u get it so What we mean by code against abstraction not concreteness?

A:

Coding against an interface: Means referencing types by their interface type (e.g., `IVehicle v = new Car();`) instead of the concrete implementation class. This ensures your code is independent of low-level details.

Coding against abstraction, not concreteness: This is a core SOLID design principle (Dependency Inversion). It means your high-level modules should rely on abstract concepts (interfaces or abstract classes) rather than specific, hard-coded concrete components, making your application extremely adaptable to future changes.

3- What is abstraction as a guideline and how we can implement this through what we have studied?

A: Abstraction as a software design guideline means focusing on essential features while hiding unnecessary complexities. We implemented this through:

Abstract Classes: Providing a shared generalized template (`GeometricShape`) with abstract requirements.

Interfaces: Exposing pure behavior contracts (`IShapeSeries`, `IVehicle`) without exposing underlying implementation logic.

4- What is operator overloading?

A: Operator overloading allows developers to redefine how standard operators (like `+`, `-`, `==`) behave when applied to custom user-defined classes or structs. For example, you can overload the `+` operator inside a `Vector` class so that adding two vector objects together calculates their mathematical sum directly (`v1 + v2`).